CMSC131: Two-Dimensional Arrays and Matrices

Chapter 10

By Naren Ayinala

May 2026

Contents

1 Why Do We Need Two-Dimensional Arrays? 3

2 Reading Rows and Columns 3

2.1 The Grid View . . . 3

2.2 The Actual Java Structure: An Array of Arrays . . . 4

3 Declaring, Creating, and Initializing a 2D Array 4

3.1 Declaration Versus Creation . . . 4

3.2 Array Literals and Rows of Different Lengths . . . 4

4 Lengths: Why `array.length` Is Not the Number of Columns 5

5 Rectangular Versus Jagged Arrays 5

5.1 Rectangular: Every Row Has the Same Length . . . 5

5.2 Jagged: Different Rows May Have Different Lengths . . . 5

5.3 Creating Jagged Rows Manually . . . 6

6 Traversing Every Element with Nested Loops 6

6.1 A Trace of the Nested Loops . . . 7

6.2 Enhanced For Loops . . . 7

7 Changing Elements and Replacing Entire Rows 7

8 Array References, Aliasing, and Copies 8

8.1 Assigning a 2D Array Does Not Copy It . . . 8

8.2 A Shallow Copy Copies Row References . . . 8

8.3 Making a Deep Copy of an `int[][]` . . . 8

9 2D Arrays and Mathematical Matrices **9**

10 Matrix Transpose: Swap Rows and Columns **9**

10.1 What Does Transpose Mean? 9

10.2 Writing a Transpose Method 9

10.3 Trace the Assignment `result[c][r] = matrix[r][c]` 10

11 Common 2D Array Algorithms **10**

11.1 Sum Every Element 10

11.2 Find the Largest Value 10

11.3 Sum Each Row Separately 11

11.4 Sum a Column in a Rectangular Matrix 11

12 Printing the Contents of a 2D Array **11**

13 2D Arrays with Objects **12**

14 Putting It Together: A Classroom Gradebook **12**

15 A Few Errors Worth Understanding **12**

16 Final Mental Model **13**

1 Why Do We Need Two-Dimensional Arrays?

A one-dimensional array is a line of values. Imagine storing the quiz scores of one student: `{90, 80, 100}`. Now imagine a classroom with three students, each taking three quizzes. We could create three separate arrays, but it would be more convenient to group them together. A two-dimensional array lets us organize values using *two indices*: one for the row and another for the column.

The phrase "two-dimensional" describes how we *use* the data, not a special new kind of memory object. In Java, a two-dimensional array is actually **an array whose elements are references to other arrays**. This detail explains rectangular grids, irregular rows, copying, and many common surprises.

```
int[][] scores = {
    {90, 80, 100},
    {75, 95, 85},
    {88, 92, 78}
};
System.out.println(scores[1][2]); // 85
```

We read `scores[1][2]` as "row 1, column 2." Both indices begin at zero, so this is the *second* row and *third* column.

2 Reading Rows and Columns

2.1 The Grid View

The example below is the same array as above. The labels across the top show column indices; the labels at the left show row indices.

	column 0	column 1	column 2
row 0	90	80	100
row 1	75	95	85
row 2	88	92	78

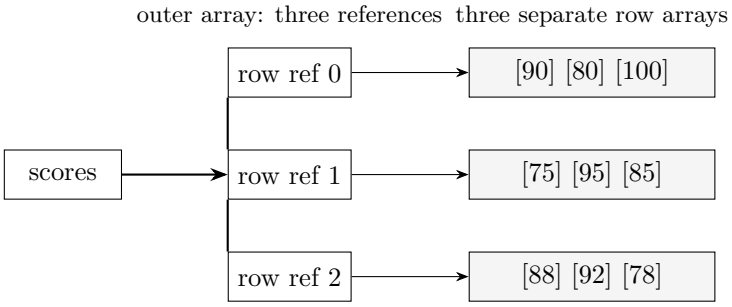
To access a cell, write `array[row][column]`. The first brackets select a row array. The second brackets select one element *inside that row*.

```
System.out.println(scores[0][0]); // 90
System.out.println(scores[0][2]); // 100
System.out.println(scores[2][1]); // 92
scores[1][0] = 79;
System.out.println(scores[1][0]); // 79
```

There is no special `scores[3]` row in this example: legal row indices are 0, 1, and 2. Each existing row has legal column indices 0, 1, and 2.

2.2 The Actual Java Structure: An Array of Arrays

The grid is a useful drawing, but the reference diagram below is closer to Java's object model. The variable `scores` points to an outer array with three slots. Each slot refers to a separate `int[]` row array.



Conceptually, `scores[1]` is an `int[]` array, and `scores[1][2]` is an `int` value. This is why the following is valid:

```
int[] secondStudent = scores[1];
System.out.println(secondStudent[2]); // 85
```

`secondStudent` and `scores[1]` now refer to the same row array. Changing one changes what the other observes.

3 Declaring, Creating, and Initializing a 2D Array

3.1 Declaration Versus Creation

```
int[][] table; // Declare a reference variable.
table = new int[2][4]; // Create two rows, four columns each.
int[][] another = new int[3][2];
```

`new int[2][4]` creates an outer array of length 2 and two inner arrays, each of length 4. All eight `int` cells start at zero. Like ordinary arrays, Java initializes elements to their type's default value.

```
int[][] nums = new int[2][3];
System.out.println(nums[0][2]); // 0
boolean[][] flags = new boolean[2][2];
System.out.println(flags[1][0]); // false
String[][] names = new String[2][2];
System.out.println(names[0][0]); // null
```

An array's elements have default values, but a local variable must still be assigned before it is used.

3.2 Array Literals and Rows of Different Lengths

```
int[][] rectangle = {
    {1, 2, 3},
    {4, 5, 6}
};
```

```
int[][] uneven = {
    {1, 2, 3, 4},
    {5},
    {6, 7}
};
```

Both are legal Java. The first is rectangular; the second is jagged (or ragged). We will examine this distinction shortly.

4 Lengths: Why `array.length` Is Not the Number of Columns

A two-dimensional array has an outer length, and *each row* has its own length. Because the rows are separate arrays, you must ask the particular row how many elements it contains.

```
int[][] data = {{3, 4, 5}, {8, 9}};
System.out.println(data.length); // 2 rows
System.out.println(data[0].length); // 3 columns in row 0
System.out.println(data[1].length); // 2 columns in row 1
```

`data.length` is not 5 (the total number of values) and it is not 3 (the first row's length). It is 2, because the *outer* array has two slots.

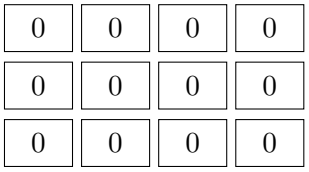
Expression	Meaning
`data.length`	Number of row references in the outer array
`data[r]`	The row array at index `r`
`data[r].length`	Number of elements in that particular row
`data[r][c]`	The element at row `r`, column `c`

5 Rectangular Versus Jagged Arrays

5.1 Rectangular: Every Row Has the Same Length

In a rectangular array, each row has the same number of columns. This resembles the rows and columns of a mathematical matrix.

```
int[][] board = new int[3][4];
// 3 rows, each with 4 columns.
```

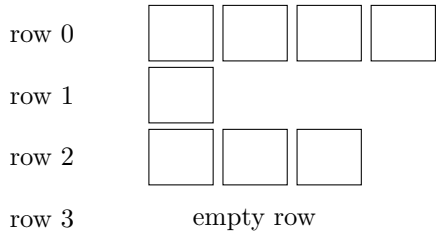


3 rows × 4 columns

5.2 Jagged: Different Rows May Have Different Lengths

Java does *not* require every row to have the same length. Because each row is an independent array, one row can have four elements, another can have one, and another can be empty.

```
int[][] jagged = {
    {10, 20, 30, 40},
    {50},
    {60, 70, 80},
    {}
};
System.out.println(jagged.length); // 4
System.out.println(jagged[1].length); // 1
System.out.println(jagged[3].length); // 0
```



Sometimes people call a rectangular array *homogeneous* in shape and a jagged array *nonhomogeneous* in shape. Be careful with that wording: **both arrays are homogeneous in element type**. Every element of an `int[][]` row is an `int`. What differs is the *row length*, not the element type. Java is statically typed; you cannot place a `String[]` into a slot that requires an `int[]`.

5.3 Creating Jagged Rows Manually

We can create only the outer array first. Its slots initially contain `null`, since each slot is an array reference.

```
int[][] rows = new int[3][];
System.out.println(rows[0]); // null
rows[0] = new int[4];
rows[1] = new int[1];
rows[2] = new int[3];
rows[0][2] = 99;
```

Trying `rows[0][0]` before assigning `rows[0]` would cause a `NullPointerException`. Trying `rows[1][1]` after assignment would cause an `ArrayIndexOutOfBoundsException`, because row 1 has only index 0.

6 Traversing Every Element with Nested Loops

A two-dimensional array is usually traversed using one loop for rows and another loop for the columns *within the current row.*

```
int[][] numbers = {{1, 2, 3}, {4, 5}, {6}};
for (int row = 0; row < numbers.length; row++) {
    for (int col = 0; col < numbers[row].length; col++) {
        System.out.print(numbers[row][col] + " ");
    }
    System.out.println();
}
```

```
// 1 2 3
// 4 5
// 6
```

The outer loop selects row 0. The inner loop prints all elements in that row, then finishes. The outer loop moves to row 1 and the inner loop starts again at column 0. Using `numbers[row].length` instead of `numbers[0].length` is essential for jagged arrays.

6.1 A Trace of the Nested Loops

Row index	Column indices visited	Values printed
0	0, 1, 2	1, 2, 3
1	0, 1	4, 5
2	0	6

A helpful mental model is: *for each row, visit each column that exists in that row.* Never assume a jagged array has the same number of columns everywhere.

6.2 Enhanced For Loops

Because a 2D array is an array of arrays, the outer enhanced `for` loop can give us each `int[]` row directly.

```
for (int[] row : numbers) {
    for (int value : row) {
        System.out.print(value + " ");
    }
    System.out.println();
}
```

This is convenient when we only need to read values. When we need to update specific cells or know their indices, indexed loops are usually clearer.

7 Changing Elements and Replacing Entire Rows

We can change one element or replace an entire row array. These are different operations.

```
int[][] values = {{1, 2}, {3, 4}};
values[0][1] = 99; // Change a cell.
values[1] = new int[]{7, 8, 9}; // Replace an entire row.
System.out.println(values[1].length); // 3
```

After replacing row 1, the outer array still has two rows, but the second row now has three columns. This is another consequence of the array-of-arrays structure.

8 Array References, Aliasing, and Copies

8.1 Assigning a 2D Array Does Not Copy It

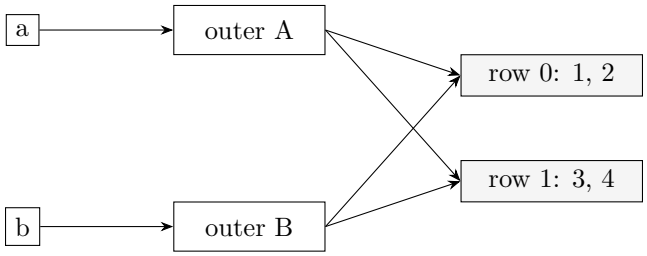
```
int[][] a = {{1, 2}, {3, 4}};
int[][] b = a;
b[0][0] = 50;
System.out.println(a[0][0]); // 50
```

`a` and `b` refer to the same outer array, which points to the same row arrays. No new array was created by `b = a`.

8.2 A Shallow Copy Copies Row References

```
int[][] a = {{1, 2}, {3, 4}};
int[][] b = a.clone();
b[0][0] = 77;
System.out.println(a[0][0]); // 77
b[0] = new int[]{8, 9};
System.out.println(a[0][0]); // still 77
```

Here the outer arrays are different, but initially they both refer to the *same inner row arrays*. Replacing `b[0]` changes only `b`'s row reference, whereas modifying `b[0][0]` before replacement changes a shared row object.



8.3 Making a Deep Copy of an `int[][]`

For independent rows, copy each row separately. The following works for jagged arrays too, provided none of the rows is `null`.

```
import java.util.Arrays;
int[][] original = {{1, 2}, {3}, {4, 5, 6}};
int[][] copy = new int[original.length][];
for (int r = 0; r < original.length; r++) {
    copy[r] = Arrays.copyOf(original[r], original[r].length);
}
copy[0][0] = 100;
System.out.println(original[0][0]); // 1
```

This is a deep copy of the *primitive row arrays*. If a 2D array instead contains references to mutable objects, copying each row still shares those objects unless you copy them as well.

9 2D Arrays and Mathematical Matrices

A mathematical matrix is a rectangular arrangement of numbers. A rectangular `int[][]` is a natural way to represent one in Java. A matrix with m rows and n columns is called an $m \times n$ matrix.

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \qquad \text{is a } 2 \times 3 \text{ matrix.}$$

We could represent it as:

```
int[][] A = {{1, 2, 3}, {4, 5, 6}};
```

A mathematical matrix must have the same number of entries in each row. A jagged Java array can still be useful for other kinds of data, but it is not a conventional rectangular matrix.

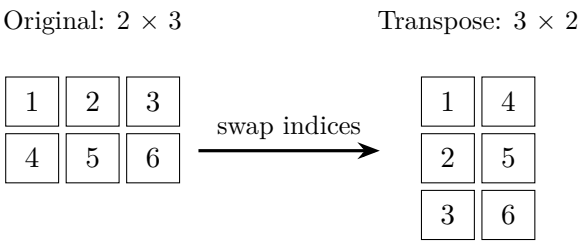
10 Matrix Transpose: Swap Rows and Columns

10.1 What Does Transpose Mean?

The **transpose** of a matrix turns rows into columns and columns into rows. The element originally at position `[r][c]` moves to `[c][r]`. We write the transpose of A as A^T.

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \quad \longrightarrow \quad A^T = \begin{bmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{bmatrix}.$$

Notice that the original matrix has 2 rows and 3 columns, while its transpose has 3 rows and 2 columns. The value 6 moves from `A[1][2]` to `transpose[2][1]`.



10.2 Writing a Transpose Method

First create an output array with reversed dimensions. Then visit each cell of the original matrix and place it at the swapped indices.

```
public static int[][] transpose(int[][] matrix) {
    int rows = matrix.length;
    int cols = matrix[0].length;
    int[][] result = new int[cols][rows];

    for (int r = 0; r < rows; r++) {
        for (int c = 0; c < cols; c++) {
            result[c][r] = matrix[r][c];
```

```
        }
    }
    return result;
}
```

This version assumes a **nonempty rectangular matrix** with at least one row. It does not work as written for an empty outer array or jagged rows. Those cases require an explicit policy or additional validation.

```
int[][] matrix = {{1, 2, 3}, {4, 5, 6}};
int[][] result = transpose(matrix);
// result is {{1, 4}, {2, 5}, {3, 6}}
```

10.3 Trace the Assignment `result[c][r] = matrix[r][c]`

Original row r	Original col c	Value	Destination
0	0	1	`result[0][0]`
0	1	2	`result[1][0]`
0	2	3	`result[2][0]`
1	0	4	`result[0][1]`
1	1	5	`result[1][1]`
1	2	6	`result[2][1]`

You are not reversing the order of values. You are swapping each element's row and column coordinates.

11 Common 2D Array Algorithms

11.1 Sum Every Element

```
int[][] data = {{2, 4}, {6}, {8, 10, 12}};
int total = 0;
for (int r = 0; r < data.length; r++) {
    for (int c = 0; c < data[r].length; c++) {
        total += data[r][c];
    }
}
System.out.println(total); // 42
```

11.2 Find the Largest Value

For a nonempty array with at least one value, initialize the maximum using a real element rather than assuming the answer must be positive.

```
int[][] data = {{-8, -3}, {-12, -5}};
int max = data[0][0];
for (int[] row : data) {
    for (int value : row) {
        if (value > max) {
```

```
            max = value;
        }
    }
}
System.out.println(max); // -3
```

11.3 Sum Each Row Separately

```
int[][] data = {{1, 2, 3}, {4, 5}, {6}};
for (int r = 0; r < data.length; r++) {
    int rowSum = 0;
    for (int c = 0; c < data[r].length; c++) {
        rowSum += data[r][c];
    }
    System.out.println("Row " + r + ": " + rowSum);
}
// Row 0: 6
// Row 1: 9
// Row 2: 6
```

11.4 Sum a Column in a Rectangular Matrix

For a rectangular matrix, we can keep the column fixed while changing the row.

```
int[][] a = {{1, 2, 3}, {4, 5, 6}};
int column = 1;
int sum = 0;
for (int r = 0; r < a.length; r++) {
    sum += a[r][column];
}
System.out.println(sum); // 7 = 2 + 5
```

For a jagged array, the chosen column might not exist in every row. Check `column < a[r].length` before accessing it, or define how missing entries should be handled.

12 Printing the Contents of a 2D Array

For one-dimensional arrays, `Arrays.toString()` works well. For nested arrays, use `Arrays.deepToString()` to show the contents of each row.

```
import java.util.Arrays;
int[][] data = {{1, 2}, {3, 4}};
System.out.println(Arrays.toString(data));
// Shows row-array reference descriptions, not the grid.
System.out.println(Arrays.deepToString(data));
// [[1, 2], [3, 4]]
```

Likewise, `Arrays.deepEquals(a, b)` compares nested array contents, while `Arrays.equals(a, b)` on two `int[][]` arrays compares their row references rather than comparing all the integers inside those rows.

13 2D Arrays with Objects

The outer/inner structure is not limited to numbers. We can store Strings or references to our own classes.

```
String[][] seats = {
    {"Ada", "Ben"},
    {"Cal", null}
};
System.out.println(seats[0][1]); // Ben
```

In `String[][]`, each inner row is a `String[]` array. Each cell of that row holds a `String` reference, which can be `null`. The outer array references row arrays, and the row arrays reference String objects. This is another level of indirection compared with `int[][]`.

14 Putting It Together: A Classroom Gradebook

Suppose three students took different numbers of quizzes. A jagged array represents this naturally. We can print each student's average without pretending everyone took the same number of quizzes.

```
int[][] quizzes = {
    {90, 100, 80},
    {75, 85},
    {100, 90, 95, 85}
};
for (int student = 0; student < quizzes.length; student++) {
    int sum = 0;
    for (int quiz = 0; quiz < quizzes[student].length; quiz++) {
        sum += quizzes[student][quiz];
    }
    double average = (double) sum / quizzes[student].length;
    System.out.println("Student " + student + ": " + average);
}
// Student 0: 90.0
// Student 1: 80.0
// Student 2: 92.5
```

The cast to `double` ensures that division does not truncate. This example assumes every student has at least one quiz; if a row were empty, we would need to decide what its average should mean before dividing.

15 A Few Errors Worth Understanding

```
int[][] a = {{1, 2}, {3}};
// a[2][0] // Invalid row index: only 0 and 1 exist.
// a[1][1] // Invalid column: row 1 has length 1.
// a.length() // Invalid: array length is a field, not a method.
// a[0, 1] // Invalid: use two pairs of brackets.
// a[0][0:2] // Invalid: Java has no Python-style slice syntax.
```

Java does not support Python's `a[0][0:2]` slice notation. If you want part of one row, use `Arrays.copyOfRange(a[0], start, end)`. Remember that `end` is excluded.

```
int[] part = Arrays.copyOfRange(a[0], 0, 2);
System.out.println(Arrays.toString(part)); // [1, 2]
```

16 Final Mental Model

A 2D array is *not* one magical rectangular block. It is an outer array containing references to row arrays. Once you remember that, the rest follows: the outer array has its own `length`; each row has its own `length`; row lengths may differ; rows may be replaced; references can be shared; and a transpose of a rectangular matrix is created by swapping the row and column indices.